

Agent 2 — Fast MVH with K-d Trees: Integration-Layer Audit, Process and Results

Technical memo (Maya baseline, Shahaf frontier tree, Roi heuristic tree, hybrid)

2026-09-25 — measurements at base commit 8870a82; branch barfe-aicahub-mvh-kdtree-perf-audit

Contents

Summary	1
Process chronology (what was done, in order, and why)	1
Key engineering decisions and the evidence behind them	2
Top-down vs bottom-up for the heuristic tree	2
Thresholds found	2
Final comparison	3
Where it does not win	3
Validity notes	3

Summary

- The fastest bit-identical configuration found is a **hybrid that keeps Shahaf’s frontier K-d tree but switches it on only when a measured scan-cost model says it pays, adds a full-dimensional witness to LOCALDOMCHECK, runs LOCALDOMCHECK before CHOOSEH at generation, stores all sets contiguously, and does *not* use Roi’s heuristic tree.**
- Against Maya’s L_NAMOA_DR_MVH (same compiler, admissible MVH): the ported solver L_NAMOA_DR_MVH_FAST is 1.9–2.7× faster at M = 3 and 5.1–11.0× at M = 4; the prototype of the same configuration is 26× at M = 5 and 46× at M = 6. Against the existing dual-tree V5: 2.2–3.2× (M = 3) and 3.0–5.4× (M = 4).
- Every completed row is bit-identical to Maya (solution MD5 + expansions + generations) except src V2 (not identical) and src V1 (crashes).
- Contradictions with the original hypotheses: the frontier-tree threshold X is dimension-dependent (low, about 8–32, for M ≥ 4; “never” for M = 3); a heuristic-tree threshold near 50 is not supported (in Maya’s order *lower* is better; in the reordered hybrid the tree does not pay at any tested |H(s)|); cache-layout changes had no measurable effect.
- The benchmark MVH generator used by earlier sweeps is **inadmissible** on the solver’s graph (loses up to 32 % of the Pareto front).

Process chronology (what was done, in order, and why)

1. **Audit of the code as written.** Read Maya’s L_NAMOA_DR_MVH (read-only submodule), the src solvers (L_NAMOA_KDT_CHOUSEH with variants ORIG/V1/V2/V3/V5, L_NAMOA_DR_MVH_KDT), StaticHeuristicKDTree, DynamicFrontierKDTree, and Shahaf’s code appendix. Flagged: V1 bound bug, V2 synthetic heuristic, stale max_g0, per-node allocations in choose_h_dual, 272-byte K-d nodes.
2. **Toolchain.** No MSVC/Boost on the machine; built with zig c++ (clang 21), -std=c++20 -O2 -DNDEBUG, Boost header shims. A single harness binary links Maya and every src solver so all comparisons share compiler and flags.

3. **Smoke test of existing variants** (legacy K50 MVH): confirmed V2 non-identical and V1 crash; V5/SHAHAF slower than Maya at M = 3 and only about 15 % faster at M = 4 despite 20× fewer comparisons — comparisons were not the bottleneck.
4. **Prototype**. `proto.hpp`: an exact re-implementation of Maya's loop (same Node, same OPEN comparator, same push order) with every structure pluggable and phase timers (`rdtsc`). First result: Maya's algorithm on contiguous storage alone was 3.8× faster (M = 4) — pointer chasing through `vector<vector>` and `NodePtr` fallbacks dominated.
5. **Phase profile**: CHOOSEH 40–70 %, LOCALDOMCHECK 33–35 % (full-D fallbacks 25 %).
6. **Soundness lemma**: `LOCALDOMCHECK(s, g)` is true iff some expanded path `p` at `s` satisfies `p ≤ g` in all coordinates. This licenses the witness shortcut and any exact frontier representation without changing a single decision.
7. **MVH generator bug** (different K gave different Pareto fronts: 8,592 / 11,444 / 11,510 vs the true 12,667). Wrote an admissible generator (Dijkstra on the symmetrised graph the solver searches); re-based all studies on admissible and APEX MVHs.
8. **Version 1 study** (frontier threshold): per-size cycle curves for flat vs K-d vs bucket K-d, in-situ sweeps of X, then a cost-model promotion rule.
9. **Version 2 study** (heuristic tree): linear vs index-ordered tree (top-down / bottom-up) vs Roi's spatial tree, fast path, suffix ideal point, redundancy pointers, threshold `hX`, leaf size — in three contexts (Maya order + flat T, Maya order + K-d T, reordered hybrid).
10. **Version 3 study** (hybrid): leave-one-out ablation of the combined configuration on seven instances (M = 3.6, APEX).
11. **Port** of the winning configuration to a clean `src`-style solver (staged only, not applied), verified bit-identical in the harness; final head-to-head against Maya and `src` solvers — stopped on request during M = 5.

Key engineering decisions and the evidence behind them

Full tables, CSV locations and per-mechanism discussion: `docs/AGENT2_PROGRESS_REPORT.md`, `docs/AGENT2_COMPARISON_REPORT.md`

decision	why	evidence
contiguous <code>G^Tr.cl(s)</code> , <code>G.cl(s)</code> , <code>Tr(H(s))</code> flat to K-d promotion when mean scan ≥ 64 comparisons and size ≥ 8	removes <code>vector<vector></code> / <code>NodePtr</code> indirection right X depends on M and on the query outcome mix	3.8× (M = 4), 2.9× (M = 5), identical algorithm E1/E2: M ≥ 4 flat for X in [0, 64]; M = 3 every tree 0.85–0.89×; cost model near-best everywhere, never promotes at M = 3 E1 (noise-level wall effect)
exact <code>max g_1</code> full-D witness in <code>LOCALDOMCHECK</code> <code>LOCALDOMCHECK</code> before CHOOSEH at generation redundancy pointers with antichain probe	fewer fallbacks, deterministic counters settles most bad fallbacks during the <code>Tr</code> scan both predicates are pure; the pushed node is unchanged landmark MVHs are 82–86 % <code>Tr</code> -redundant	1.03–1.36× (E4); bad fallbacks 81,285 to 207 CHOOSEH calls +4.2; 1.19–2.40× (E4) CHOOSEH comparisons +6.5; $\leq 1.09\times$ in hybrid, 1.26–1.30× in Maya order E3c/E4: tree 0.92–1.17× of FULL
Roi tree off by default	after reordering, calls are few and mostly answered by <code>h[start]</code> ; build costs 7–9 %	E3b: flat T 0.34–0.62×, K-d T 1.62×; E3c 1.2–1.9×
Roi tree on (<code>hX</code> = 0) only with Maya's order and a K-d T	failed min-box tests are cheap in a K-d T, full scans in a flat T	E1/E2/E2b/E3/E4
discarded: bucket K-d, filtered-T traversal, branchless kernel, last-witness cache, <code>G.cl</code> pruning, padded vs compact nodes	no gain or harmful	

Top-down vs bottom-up for the heuristic tree

Every successful CHOOSEH must pay one non-dominated query `Dom(T, Tr(g)+Tr(h))` to certify its answer; a tree can only save refutations. In an index-ordered tree, bottom-up search from `leaf(start)` examines only right siblings, whose boxes contain only indices $\geq \text{start}$, in index order: the first accepted box or leaf hit is the answer, and the first test is `h[start]` itself (an implicit fast path). Top-down tests ancestors whose boxes also cover the refuted prefix, so their Rule-1 tests fail more often and each failure is a full query. Measured: 1.25× vs 1.02× (M = 4), 1.29× vs 0.99× (M = 5). Roi's tree is spatial, so a bottom-up walk has no index meaning there (the ancestors of `leaf(start)` do not bound `[start, |H(s)|]`); it needs top-down traversal with a fast path, index-range pruning and children in `min_idx` order.

Thresholds found

component	hypothesis given	found
frontier tree X (V1)	build after X paths	M = 3: never (trees 0.85–0.89×); M = 4: X about 16–64 equivalent; M = 5: about 16–32; best single rule: cost model (mean scan ≥ 64 , size ≥ 8)
heuristic tree <code>hX</code> (V2)	about 50	Maya order: <code>hX</code> = 0 best (<code>hX</code> = 64 loses 3–10 %, <code>hX</code> = 1024 most of the gain); hybrid order: no <code>hX</code> pays on the tested instances
leaf size	8	8–32 equivalent, 4 worst
K-d rebuild	built/4 + 64 vs 2·built + 8	keep built/4 + 64

Final comparison

Wall-clock seconds, admissible MVH, same binary; min of 3 reps (Maya 1 rep).

instance	M	Maya	SHAHAF	ORIG	V5	FAST	FAST/Maya	FAST/V5	identical
g10M3-A50	3	0.166	0.305	0.211	0.284	0.088	1.88×	3.23×	yes
g15M3-A100	3	1.150	1.676	1.081	1.325	0.427	2.69×	3.10×	yes
g15M3-A200	3	1.387	2.852	1.246	1.614	0.731	1.90×	2.21×	yes
g10M4-A10	4	5.035	3.302	6.748	3.523	0.995	5.06×	3.54×	yes
g10M4-A25	4	7.257	4.179	8.126	3.634	0.854	8.50×	4.25×	yes
g10M4-A50	4	9.438	4.806	8.366	4.055	1.333	7.08×	3.04×	yes
g10M4-A100	4	9.613	6.819	9.944	4.105	1.019	9.44×	4.03×	yes
g12M4-A50	4	11.044	6.141	13.695	5.473	1.008	10.95×	5.43×	yes
g10M5-A50	5	77.922	23.815	76.351	–	–	–	–	yes (stopped)

Prototype FULL (same mechanisms as FAST) vs Maya reference (seconds):

instance	M	Maya	proto none (layout only)	proto FULL	FULL/Maya	identical
g10M4-A50	4	9.434	2.749	0.906	10.4×	yes
g12M4-A50	4	11.225	4.194	1.063	10.6×	yes
g10M5-A50	5	87.370	25.714	3.348	26.1×	yes
g10M6-A25	6	72.594	23.000	1.576	46.1×	yes
apex-g10M4	4	2.168	1.278	0.642	3.4×	yes
apex-g8M5	5	1.794	1.152	0.286	6.3×	yes
g15M3-A200	3	1.541	1.054	0.613	2.5×	yes

Where it does not win

- Frontier K-d trees at M = 3 are a regression (0.85–0.89×); FAST avoids trees there, so its M = 3 gain comes from layout, witness and ordering only.
- Existing src K-d solvers are slower than Maya at M = 3 (SHAHAF 0.49–0.69×, V5 0.58–0.87×).
- Roi's tree alone (ORIG) is slower than Maya on 4 of 5 M = 4 instances and at parity at M = 5.
- Roi's tree inside the hybrid does not pay (0.92–1.17× of FULL).
- Not measured in the final harness run (stopped): V5/FAST at M = 5/6, APEX and legacy instances; only one M = 6 instance.

Validity notes

- Timing on a throttling laptop: min of reps, interleaved configs, pinned core; differences below about 10 % are noise.
- Bit-identity is defined per standard library: heap tie-breaking in `std::priority_queue` may differ between MSVC and libc++, so Maya must be rebuilt with the same toolchain as any solver compared against it.
- `cmp` counters differ in scope between solvers (Maya has none); compare wall time, expansions and generations.
- No `src/` or `baselines/` file was modified; the proposed change set is `scratchpad/proposal/proposal_clean.patch`.